

Polymorphism

1. What Polymorphism Means, Concretely

Polymorphism ("many forms") — the same method call/interface behaves differently depending on **what's actually being called on**.

Two fundamentally different mechanisms achieve this, resolved at **different times**:

	Compile-Time (Static) Polymorphism	Runtime (Dynamic) Polymorphism
Resolved	At compile time	At runtime
Mechanism	Function overloading, operator overloading	Function overriding + virtual functions
Based on	The number/type of arguments, known at compile time	The actual runtime type of the object

2. Compile-Time Polymorphism: Overloading

Function overloading — multiple functions/methods share the same name but differ in their **parameter list** (number, type, or order of parameters).

```
class Printer {
public:
    void print(int x) { cout << "int: " << x; }
    void print(string s) { cout << "string: " << s; }
    void print(int x, int y) { cout << "two ints: " << x << ", " << y; }
};
```

```
class Printer {
    void print(int x) { System.out.println("int: " + x); }
    void print(String s) { System.out.println("string: " + s); }
    void print(int x, int y) { System.out.println("two ints: " + x + ", " + y); }
}
```

Key characteristics

- **Resolved at compile time** — the compiler looks at the arguments in the call site and picks the matching overload; there's no runtime decision involved at all.

- **Distinguished by the method signature** (name + parameter types/order) — **not** by return type.
- **Return type alone cannot distinguish overloads:**

```
cpp int print(int x); string print(int x); // COMPILE ERROR — same signature (name + params),  
differing only in return type
```

This is a very common interview trap — "can you overload two functions that differ only in return type?" **No.**

Interview soundbite: "Overloading isn't really about polymorphism in the deep, dynamic-dispatch sense — it's better described as the compiler picking, at compile time, which of several unrelated (but similarly-named) functions to actually call. The 'poly' here is superficial: multiple *forms* of the function exist in source, but exactly one is baked into the compiled code at each call site, permanently."

3. Runtime Polymorphism: Overriding

Function overriding — a derived class provides its **own implementation** of a method that's already defined in its base class, with the **exact same signature**.

```
class Animal {  
public:  
    virtual void sound() { cout << "Animal makes a sound"; } // VIRTUAL — enables overriding  
};  
class Dog : public Animal {  
public:  
    void sound() override { cout << "Dog barks"; } // OVERRIDES Animal::sound()  
};
```

```
class Animal {  
    void sound() { System.out.println("Animal makes a sound"); } // virtual by DEFAULT in Java  
}  
class Dog extends Animal {  
    @Override  
    void sound() { System.out.println("Dog barks"); }  
}
```

Overloading vs. Overriding — the distinction that gets tested constantly:

	Overloading	Overriding
Relationship	Same class (or unrelated)	Base class ↔ derived class
Signature	Must differ	Must be identical
Resolved	Compile time	Runtime
Purpose	Multiple ways to call a similarly-named operation	Specialize inherited behavior for a subtype

4. Static vs. Dynamic Binding — the mechanism behind overriding actually working

This is the single most commonly tested C++ interview concept in this whole area, because it's easy to get **silently wrong**.

The exact same code, differing only in one keyword:

```
// WITHOUT virtual – STATIC BINDING
class Animal {
public:
    void sound() { cout << "Animal makes a sound" << endl; }    // no `virtual`
};
class Dog : public Animal {
public:
    void sound() { cout << "Dog barks" << endl; }
};
int main() {
    Animal* animal;
    Dog dog;
    animal = &dog;
    animal->sound();      // OUTPUT: "Animal makes a sound" ← resolved by POINTER's static type!
}
```

```
// WITH virtual – DYNAMIC BINDING
class Animal {
public:
    virtual void sound() { cout << "Animal makes a sound" << endl; }    // `virtual` added
};
class Dog : public Animal {
public:
    void sound() { cout << "Dog barks" << endl; }
};
int main() {
    Animal* animal;
    Dog dog;
    animal = &dog;
    animal->sound();      // OUTPUT: "Dog barks" ← resolved by the OBJECT's actual runtime type!
}
```

Same exact code at the call site (`animal->sound()`) — two completely different outputs, purely because of one keyword.

	Static Binding	Dynamic Binding
Also called	Early binding	Late binding
Resolved using	The pointer/reference's declared (static) type	The object's actual runtime type
Trigger	Default in C++ (no <code>virtual</code>)	<code>virtual</code> keyword in C++; default in Java
When resolved	Compile time	Runtime

Interview soundbite: "Static binding asks 'what type is this variable *declared* as?' Dynamic binding asks 'what type is the object *actually*, right now, at runtime?' In C++, you opt **into** dynamic binding explicitly with `virtual` — the default is static. In Java, every non-static, non-final, non-private method is virtual by default — you'd have to go out of your way (`final`) to opt out."

5. The vtable Mechanism (how dynamic binding is actually implemented)

Each **class with virtual functions** gets a compiler-generated **virtual table (vtable)** — an array of function pointers, one per virtual function, pointing to the correct implementation **for that class**.

```
Animal's vtable: [ &Animal::sound ]
Dog's vtable:   [ &Dog::sound ]      ← overrides the entry inherited from Animal
```

Every object of a class with virtual functions carries a hidden "vptr" (vtable pointer), set at construction time, pointing to its OWN class's vtable.

```
animal->sound():
  1. Follow animal's vptr → get to the ACTUAL object's vtable (Dog's, since the real object is a Dog)
  2. Look up the "sound" slot in that vtable
  3. Call whatever function pointer is stored there → Dog::sound()
```

Why this explains the static/dynamic binding difference precisely: without `virtual`, the compiler resolves `animal->sound()` directly at compile time, based purely on the *declared* type of `animal` (`Animal*`) — it never even looks at what `animal` actually points to at runtime. With `virtual`, the compiler instead emits code to look through the **vptr → vtable → function pointer** chain at runtime — and since `animal`'s vptr was set (at construction) to point to *Dog's* vtable (because the real object is a `Dog`), it correctly finds `Dog::sound()` .

Cost of dynamic binding: one extra pointer per object (the vptr) and one extra indirection per virtual call (vptr → vtable → function) — genuinely small, but non-zero, which is exactly why C++ makes it **opt-in** via `virtual` rather than the universal default (a deliberate "don't pay for what you don't use" design philosophy) — unlike Java, which defaults to virtual dispatch everywhere and accepts that small cost universally in exchange for consistent, predictable override behavior.

6. Why Polymorphism Actually Matters — the Hierarchical Inheritance payoff

Recall from the Inheritance topic: hierarchical inheritance lets multiple classes share one base. Polymorphism is what makes that structure *useful* for writing general code:

```
class Shapes {
public:
    virtual float area() { return 0; }
};
class Square : public Shapes {
    int side;
public:
    Square(int s) : side(s) {}
    float area() override { return side * side; }
};
class Triangle : public Shapes {
    int base, height;
public:
    Triangle(int b, int h) : base(b), height(h) {}
    float area() override { return 0.5 * base * height; }
};

int main() {
    Shapes* shapes[] = { new Square(5), new Triangle(6, 10) };
    for (int i = 0; i < 2; i++)
        cout << "Area: " << shapes[i]->area() << endl;    // 25, then 30
    for (int i = 0; i < 2; i++) delete shapes[i];
}
```

The critical insight: the loop **doesn't know or care** whether each element is actually a `Square` or a `Triangle` — it just calls `area()` on a `Shapes*`, and the **correct** version runs each time, thanks to dynamic binding. Adding a `Circle` class later requires **zero changes** to this loop — it would just work. This exact property is what the **Open/Closed Principle** (see the SOLID topic) formalizes as a design goal.

Note: `Shapes` here needs a `virtual ~Shapes()` destructor too, for exactly the reason covered in the Constructors & Destructors topic — deleting through a base pointer without one would silently skip derived-class cleanup.

7. Upcasting and Downcasting

- **Upcasting** — treating a derived object as its base type (`Shapes* s = new Square(5);`) — always **safe**, always allowed implicitly, since a `Square` genuinely is a `Shapes`.
- **Downcasting** — treating a base pointer as a specific derived type (`Square* sq = static_cast<Square*>(s);` or, safely, `dynamic_cast<Square*>(s)` in C++) — **not always safe**, since the object might not actually be that derived type at all. `dynamic_cast` checks this at runtime and returns `nullptr` (or throws, for references) if the cast is invalid — the safe, checked way to downcast when you're not certain of the actual type.

Interview Questions With Answers

Q1. Why is overloading described as "compile-time polymorphism" when arguably nothing is being dispatched dynamically at all?

Answer: The term reflects that multiple candidate functions exist with the same name (giving the appearance of one name having "many forms"), but which specific one actually gets called is fully and permanently determined by the compiler at compile time, based on the argument types/count at each call site — there's no runtime decision-making or dispatch mechanism involved at all, unlike true dynamic (runtime) polymorphism. It's "polymorphism" in a much shallower sense than overriding is — the resolution just happens earlier (compile time) rather than later (runtime).

Q2. Why can't two functions be overloaded based on return type alone?

Answer: Overload resolution works by having the compiler look at a call site's arguments and match them against each overload's parameter list — but a function's return type isn't part of that matching process, and in many call contexts, the return value might not even be used or assigned anywhere, giving the compiler no reliable information to distinguish which overload was intended based on return type. Since the method signature that identifies an overload consists of the name plus parameter types/order (not return type), two functions differing only in return type are considered to have an identical signature — a redeclaration conflict, not a valid overload.

Q3. Using the Animal/Dog example, explain precisely why removing the `virtual` keyword changes the output of `animal->sound()` from "Dog barks" to "Animal makes a sound," even though `animal` still points to an actual Dog object at runtime.

Answer: Without `virtual`, the compiler resolves which `sound()` implementation to call using **static binding** — based purely on the *declared* type of the pointer (`Animal*`), determined entirely at compile time, without any regard to what the pointer might actually point to when the program runs. Since `animal` is declared as `Animal*`, the compiler binds the call directly to `Animal::sound()`, regardless of the fact that it happens to point to a `Dog` object at runtime. With `virtual`, the compiler instead generates code that looks up the correct implementation through the object's vtable at runtime — which correctly reflects the object's actual type (`Dog`), producing "Dog barks" instead.

Q4. Explain, mechanically, how the vtable and vptr work together to make dynamic binding possible.

Answer: Every class that has virtual functions gets its own compiler-generated vtable — essentially an array of function pointers, one per virtual function, each pointing to that specific class's implementation (or an inherited one, if not overridden). Every object of such a class carries a hidden pointer (the vptr), set at construction time to point to its own class's vtable. When a virtual function is called through a base-class pointer/reference, the compiler emits code that follows that object's vptr to reach its actual class's vtable, looks up the correct function pointer for the called method, and invokes it — since the vptr always points to the *actual* object's class's vtable (set correctly at construction, regardless of what

type of pointer is later used to access the object), this mechanism always resolves to the true, runtime-correct implementation.

Q5. Why does C++ make virtual dispatch opt-in (via the `virtual` keyword), while Java makes it the default for all non-final, non-private methods?

Answer: Virtual dispatch has a real, if small, runtime cost — an extra pointer per object (the `vptr`) and an extra indirection per virtual call (`vptr` → `vtable` → function) compared to a direct, statically-resolved call. C++'s design philosophy generally favors "don't pay for what you don't use" — since not every method needs to be overridable, requiring `virtual` explicitly lets a C++ program pay that cost only where dynamic dispatch is actually needed. Java's design philosophy prioritizes consistent, predictable behavior and safety over this specific micro-optimization — defaulting every method to virtual means overriding always works the way a Java developer expects without needing to remember an extra keyword, at the cost of universally accepting the small dispatch overhead.

Q6. Why does the `Shapes* shapes[]` polymorphic array example not need to change at all when a new `Circle` class is added later?

Answer: The loop iterating over `shapes[]` only ever calls `shapes[i]->area()` through the base `Shapes*` type — it has no code path that depends on knowing the concrete type of any specific element. Because `area()` is virtual, dynamic binding ensures that whatever the actual runtime type of each element is (Square, Triangle, or a newly-added Circle), the *correct*, type-specific `area()` implementation is automatically called — the loop's logic is expressed entirely in terms of the shared `Shapes` interface, so extending the family of shapes never requires touching code that only interacts with that shared interface. This is exactly the practical value polymorphism delivers: new types can be added later that seamlessly work with existing code, with zero modification to that existing code.

Q7. What is the difference between upcasting and downcasting, and why is `dynamic_cast` considered the "safe" way to downcast in C++?

Answer: Upcasting treats a derived-class object as its base type — always safe and implicit, since a derived object genuinely is an instance of its base type by the nature of inheritance. Downcasting treats a base-class pointer/reference as a more specific derived type — this is not always safe, because the pointer's declared base type doesn't guarantee the actual underlying object really is that specific derived type. `dynamic_cast` is safe because it performs an actual runtime check (using the same RTTI/vtable-adjacent information that supports dynamic binding) to confirm whether the object truly is the target derived type before allowing the cast to succeed — returning `nullptr` (for pointers) or throwing an exception (for references) if the object isn't actually that type, rather than silently producing an invalid pointer that would cause undefined behavior if used, as a plain `static_cast` downcast would risk doing without any such check.

Q8. Scenario: A colleague writes a base class `Shape` with a non-virtual `draw()` method, reasoning "I'll add `virtual` later if I ever actually need polymorphism, to avoid the small runtime overhead until then." They then write derived classes `Circle` and

`Square`, each overriding `draw()`, and store a mix of them in a `vector<Shape*>`, calling `->draw()` on each through the base pointer. What will actually happen, and why is the colleague's reasoning flawed?

Answer: Every call to `->draw()` through the `Shape*` pointers will resolve, via static binding, to `Shape::draw()` — regardless of whether each pointer actually points to a `Circle` or a `Square` object — because without the `virtual` keyword, the compiler binds the call based on the pointer's declared type (`Shape*`) at compile time, never checking the object's actual runtime type at all. This directly contradicts what the colleague is trying to achieve (`Circle` and `Square` objects behaving differently through the shared base pointer) — their "derived overrides" are actually just separate, unrelated functions that happen to share a name, never actually invoked through the polymorphic array at all. The reasoning is flawed because "adding virtual later, when polymorphism is needed" doesn't work retroactively in the way they expect — polymorphic behavior through base-class pointers requires `virtual` to be present on the base class method from the moment that polymorphic usage pattern (storing/calling through base pointers, expecting derived-specific behavior) is actually written; the overhead concern is also largely moot here, since they're already committing to `Shape*` pointers and heap allocation, and the vtable/vptr overhead is negligible compared to those costs.